Zephyr Project

IT-Schulungen.com
Technology & Transformation

Timer und Workqueues

Auslösen, aufschieben und ausführen

2.12

Vom Zeitpunkt zur Verarbeitung

01	`Timer läuft ab`	Ein Zeitpunkt oder eine Periode löst ein kurzes Ereignis aus
02	`Arbeit einreichen`	Der Expiry-Handler übergibt die Aufgabe an eine Workqueue
03	`Handler arbeitet`	Ein Workqueue-Thread führt die aufgeschobene Verarbeitung aus

Auslöser und Verarbeitung bleiben getrennt: zeitkritisch kurz, umfangreiche Arbeit später.

Der Ausführungskontext entscheidet

TIMER

Auslöser

Kurz und nicht blockierend

Zeitpunkt oder Periode melden

WORKQUEUE

Aufgeschobene Arbeit

Läuft in einem Thread

Teilt dessen Priorität und Kapazität

EIGENER THREAD

Dauerhafter Ablauf

Eigener Stack und Zustand

Darf gezielt blockieren

Wählt den Kontext nach Verantwortung, Blockierverhalten und benötigter Unabhängigkeit.

Timer übergibt an Workqueue

K_TIMER + K_WORK

```
K_WORK_DEFINE(sample_work, sample_handler);

static void expiry(struct k_timer *timer)
{
    k_work_submit(&sample_work);
}

K_TIMER_DEFINE(sample_timer, expiry, NULL);
```

01 Expiry

Nur Ereignis erfassen und Arbeit einreichen.
Nicht warten, nicht aufwendig verarbeiten.

02 Work-Handler

Läuft später im Workqueue-Thread.
Bearbeitet die eigentliche Aufgabe.

Der Timer bestimmt wann – der Work-Handler bestimmt was.

Workqueue-Regeln bewusst anwenden

`System-Workqueue`	Bequem, aber von allen Nutzern gemeinsam verwendet
`Langer Handler`	Blockiert nachfolgende Arbeit in derselben Queue
`Delayable Work`	Aufgabe für später planen oder erneut terminieren
`Bereits anstehend`	Status und Rückgabewert bei erneuter Einreichung beachten
`Eigene Workqueue`	Isolation wählen, wenn Priorität oder Laufzeit unabhängig sein muss

Work-Handler bleiben kurz, begrenzt und dürfen die gemeinsame Queue nicht monopolisieren.

Mechanismus nach Aufgabe wählen

ENTSCHEIDUNG

Welche Verantwortung braucht einen eigenen Ausführungskontext?

`k_timer`	Zeitpunkt oder periodischen Impuls auslösen
`System-Workqueue`	Kurze, unkritische Arbeit gemeinsam ausführen
`Eigene Workqueue`	Aufgeschobene Arbeit mit eigener Priorität isolieren
`Eigener Thread`	Dauerhaften oder blockierenden Ablauf steuern

Timer löst aus; Workqueue verschiebt Arbeit; Thread trägt dauerhafte Ablaufverantwortung.

Kurzcheck: Timer und Workqueues

AUSLÖSER

Timer

kurz reagieren

nicht blockieren

Arbeit weiterreichen

Periode bewusst wählen

ARBEIT

Workqueue

Handler begrenzen

Rückgabewert prüfen

System-Queue teilen

Isolation bei Bedarf

ABLAUF

Thread

eigener Stack

darf blockieren

eigener Zustand

klare Verantwortung

Transferfrage: Wo gehört eine längere, potenziell blockierende Verarbeitung hin – und warum?